DHBW
Duale Hochschule
Baden-Württemberg

Dokumentation zur Entwicklung eines Terminplanungssystem

Dokumentation

Studiengang Informatik

Duale Hochschule Baden-Württemberg Stuttgart

Felix Hennerich

Bruno Lange

Eingereicht am:	30.10.2026
Matrikelnummer, Kurs:	2594549, TINF24F 4086963, TINF24F
Dozent an DHBW:	Lucas Späth

Inhalt

1 Einleitung **1**
1.1 Ziel . . . 1
1.2 Architektur-Schaubild . . . 1

2 Threat Modeling **3**
2.1 Data Flow Diagramm . . . 3
2.2 STRIDE . . . 4
2.2.1 STRIDE Datenbank . . . 4
2.2.2 STRIDE Backend . . . 5
2.2.3 STRIDE FRONTEND . . . 6
2.3 Attack Tree . . . 7

3 Sicherheitsfeatures **8**

4 **9**

5 Deployment **10**
5.1 Prozess . . . 10
5.2 CI/CD Pipeline . . . 10

6 Lessons-Learned **11**

A Literatur **12**

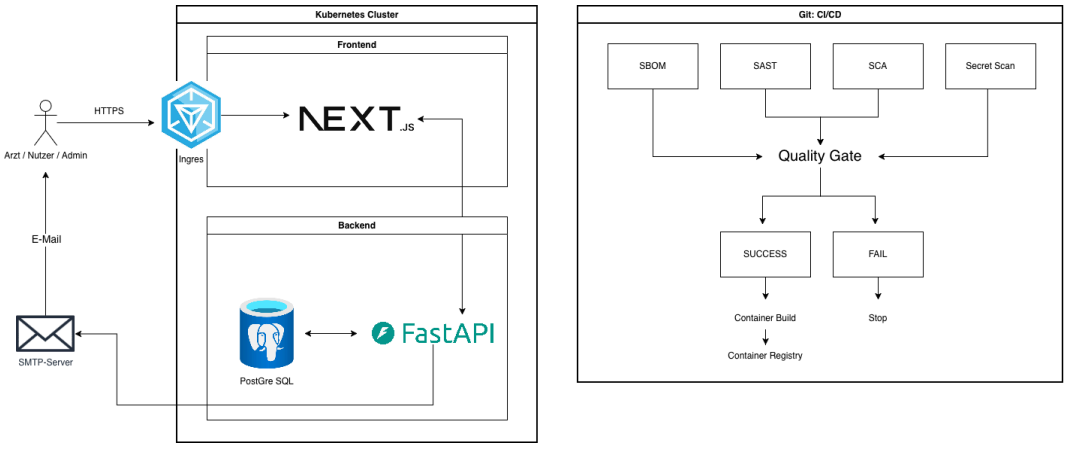
B Glossar **13**

1 Einleitung

1.1 Ziel

Ziel dieses Entwicklungsprojektes ist die Entwicklung eines digitalen Terminplanungstools für Arztpraxen. Die Software soll sowohl Patienten als auch Mitarbeitern den Praxisalltag erleichtern und bestehende Abläufe effizienter gestalten. Patienten sollen künftig die Möglichkeit haben, eigenständig freie Termine online zu buchen. Dadurch soll insbesondere das Telefonaufkommen in der Praxis reduziert und das Rezeptionspersonal entlastet werden, sodass dieses mehr Zeit für andere Aufgaben hat. Darüber hinaus soll die Software den Mitarbeitern eine Terminverwaltung ermöglichen. Hierfür werden entsprechende Funktionen für die Rezeptions- bzw die Doktoransicht bereitgestellt, über die Termine geplant und verwaltet werden können.

1.2 Architektur-Schaubild



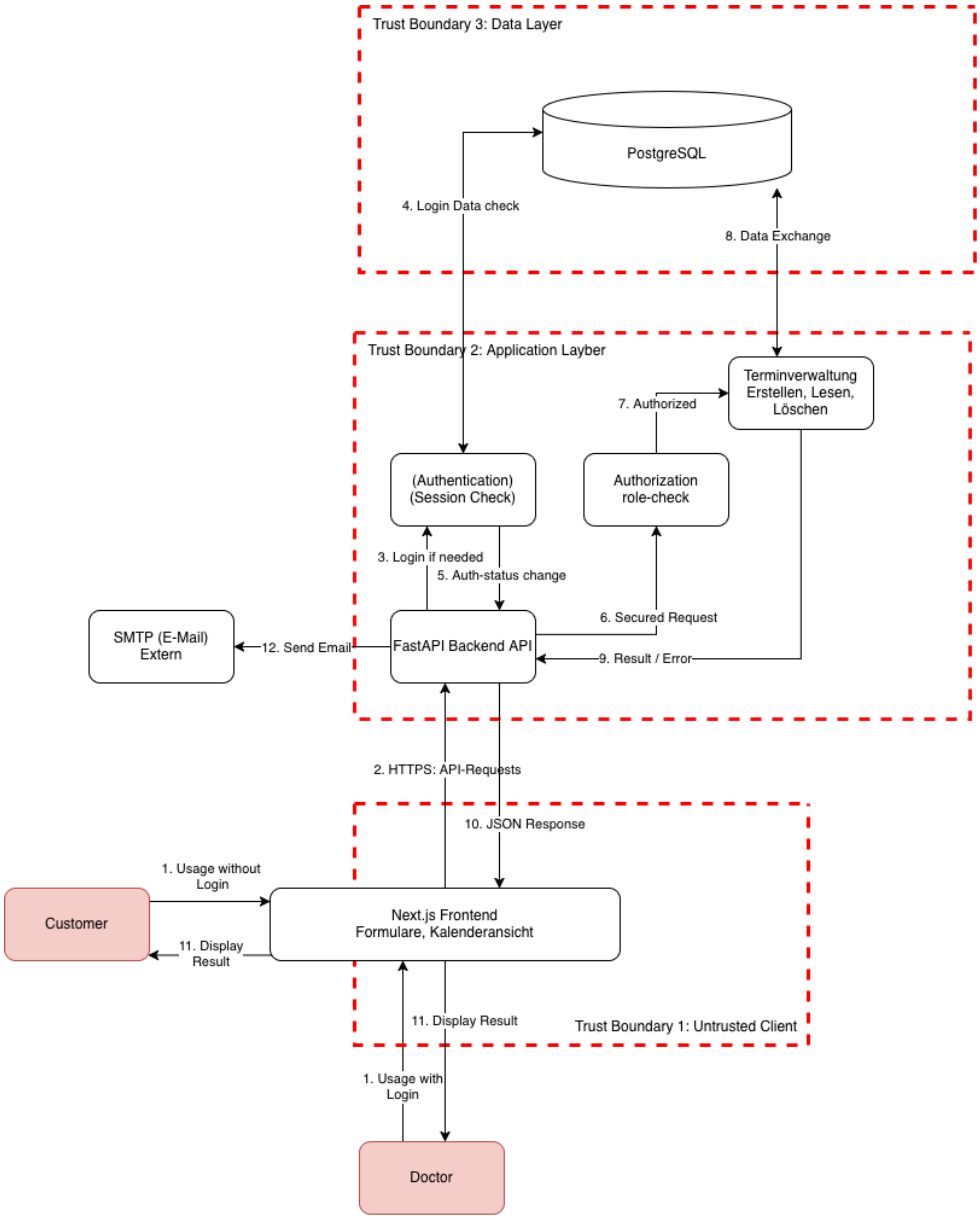
- **Next.js:** React-basiertes Frontend-Framework auf TypeScript-Basis. Es wird zur Entwicklung der grafischen Benutzeroberfläche verwendet.
- **FastAPI:** Python-basiertes Backend-Framework zur Bereitstellung der REST-API. Es übernimmt den Datenaustausch zwischen Frontend und Datenbank sowie die Verarbeitung und Aufbereitung der Daten.

- **PostgreSQL:** Relationales Datenbankmanagementsystem zur Speicherung und Verwaltung der Anwendungsdaten.
- **Arzt / Patient / Administrator:** Das Berechtigungskonzept der Software ist bewusst einfach gehalten. Patienten benötigen keinen eigenen Account, um Termine buchen zu können. Dadurch wird die Einstiegshürde für die Terminvereinbarung reduziert. Die Anmeldung ist primär für die internen Benutzer der Praxis vorgesehen.
- **SMTP:** Das SMTP wird für den Versand automatisierter E-Mails verwendet. Patienten erhalten beispielsweise nach einer erfolgreichen Terminbuchung eine Bestätigungs-E-Mail.
- **Kubernetes:** Kubernetes wird zur Orchestrierung und Verwaltung der containerisierten Anwendungsbestandteile eingesetzt. Es ermöglicht die automatisierte Bereitstellung, Skalierung und Überwachung der einzelnen Dienste.

2

Threat Modeling

2.1 Data Flow Diagramm



2.2 STRIDE

2.2.1 STRIDE Datenbank

STRIDE	Bedrohung	Beschreibung	Beispielangriff	Voraussetzungen	Gegenmaßnahmen
S	Anmeldung mit Priviligiertem Account	Dev-Account komprimitieren und damit anmelden.	Default Passwörter testen oder Credential Stuffing	Kein Mehrfaktor oder geblockter Netzwerkzugang	Firewall
T	Manipulation von Daten via SQL-Anfragen	Durch SQL Injection können Querys geändert werden	SQL Injection (`OR 1=1 -- bei login UI)	Verwundbare SQL Anfragen / Verwundbare Inputfelder	Prepared Statements nutzen beim Entwickeln
R	Benutzer löscht Daten ohne diese Nachzuvollziehen	Löschung von Termin ohne nachverfolgung wer gelöscht hat	Ein Administrator löscht Datensätze und bestreitet anschließend die Änderung, da keine geeigneten Audit-Protokolle existieren.	Fehlende oder unzureichende Protokolle / Logs	PostGres-Logging, klar ausgearbeitetes Logging Konzept
I	Unbefugtes Erhalten von Nutzerdaten wie Emails	Nutzer A kann irgendwie Daten von Nutzer B erlangen	Nutzer geben über SQL-Injections mehr daten aus als tatsächlich benötigt	Falsche Zugriffsbeschränkung oder verwundbare SQL Anfragen	Least-Privilege-Prinzip und Nutzung von Prepared Statements
D	Überlastung und Ausfall der Datenbank	Datenbank geht offline sodass die Software beeinträchtigt wird	Erzeugung zahlreicher unnötiger SQL Anfragen um DB zu überlasten	Fehlende Ressourcen oder Lastbegrenzung	Verbindungs und Abfrage Limits, IP-Blocking bei vielen Anfragen
E	Erlangen von höheren Rechten	Nutzer kann seine Rechte erweitern sodass man an sensible Inhalte gelangt	Neue Nutzer mit mehr rechten anlegen in der Datenbank	Fehlkonfigurierte Rollen, unsichere Funktionen	Least-Privilege-Prinzip, keine unnötige Vergabe von Super-User-Rechten

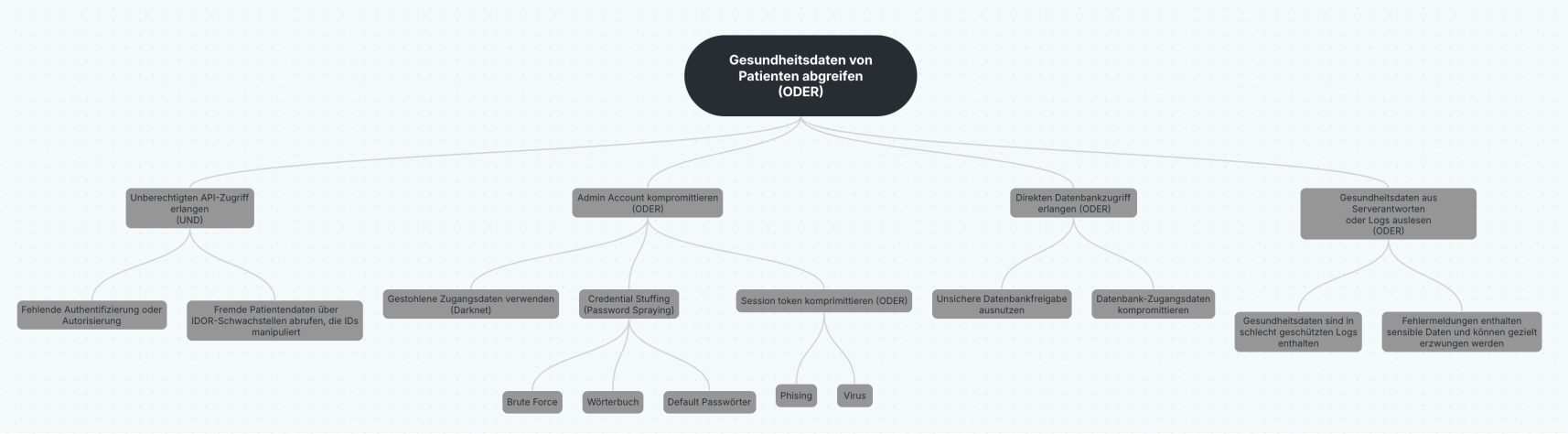
2.2.2 STRIDE Backend

STRIDE	Beschreibung	Beispielangriff	Voraussetzungen	Gegenmaßnahmen
S	Vortäuschen einer Benutzeridentität	Verwendung einer gestohlenen Session / Manipulation eines Session-Cookies	Fehlende Session-Validierung / Gestohlene Zugangsdaten	Serverseitige Session-Validierung / Sichere Session-Tokens / Cookies mit HttpOnly, Secure, SameSite
T	Unberechtigte Manipulation von API-Anfragen oder Daten	Änderung einer fremden Termin-ID / Manipulation von Termindaten	unzureichende Eingabevalidierung / Fehlende Objektberechtigungen	Parametrisierte SQL-Abfragen / Serverseitige Berechtigungsprüfung
R	Aktionen können nicht eindeutig nachvollzogen werden	Arzt bestreitet das Löschen eines Termins	Fehlende oder manipulierbare Audit-Logs	Audit-Logging / Zeitstempel und Nutzer-ID protokollieren / Manipulationsgeschützte Logs
I	Offenlegung vertraulicher Daten	Abruf fremder Patientendaten / Ausgabe interner Stacktraces	Fehlende Zugriffskontrollen / Ausführliche Fehlermeldungen	Zugriffskontrollen für alle geschützten Endpunkte / Minimale API-Antworten / Generische Fehlermeldungen / Least-Privilege-Prinzip
D	Beeinträchtigung der Verfügbarkeit des Backends	Massenhafte API-Anfragen / Überlastung der Datenbank durch ressourcenintensive Abfragen	Fehlende Rate Limits / Unbegrenzte Request-Größen / Fehlende Timeouts	Rate Limiting / Request-Größen begrenzen / Timeouts konfigurieren / Datenbankabfragen optimieren
E	Unberechtigte Erweiterung von Nutzerrechten	Patient löscht einen fremden Termin / Zugriff auf Arztfunktionen ohne entsprechende Rolle	Fehlende oder fehlerhafte Rollenprüfung / Vertrauen in Client-seitige Rollenangaben	Berechtigungsprüfung pro Anfrage und Ressource / Standardmäßig Zugriff verweigern

2.2.3 STRIDE FRONTEND

STRIDE	Beschreibung	Beispielangriff	Voraussetzungen	Gegenmaßnahmen
S	Nutzung einer Benutzeridentität einer anderen Person	Anzeige einer gefälschten Login-Seite	Nutzer fällt auf Phishing rein	MFA beispielsweise TOTP
T	Manipulation von Formulareingaben	XSS Angriffe in Eingabefelder	Eingabefelder haben keine Inputvalidation auf XSS	Implementierung von XSS Sicherung
R	Benutzeraktionen können nicht nachvollzogen werden	Nutzer bestreitet das Absenden oder Ändern eines Termins	Fehlende oder Falsche Protokollierung relevanter Aktionen	Audit-Logging im Backend / Timestamp und Nutzer-ID erfassen
I	Leak von sensiblen Daten im Frontend	Anzeige fremder Patientendaten	Fehlende oder Falsche Zugriffskontrollen	Zugriffskontrollen richtig implementierung und testen
D	Überlastung des Webservers	Tausendfaches neuladen der unterschiedlichen Unterseiten	Fehlendes oder falsch Konfiguriertes Rate Limiting	Implementierung von Rate Limiting im Reverse Proxy
E	Umgehung der Rollenbeschränkung auf Unterwebseiten	Patient ruft eine nur für Ärzte vorgesehene Seite direkt über die URL auf	Rollenprüfung auf jeder Unterseite sowie serverseitige Zugriffskontrolle	Geschützte Seiten serverseitig prüfen / Berechtigungen im Backend erzwingen

2.3 Attack Tree



3

Sicherheitsfeatures

Erklärung der Anwendung mit allen Sicherheitsfeatures

4

Erklärung aller nicht-sicheren Parts mit Begründung

5

Deployment

5.1 Prozess

5.2 CI/CD Pipeline

6

Lessons-Learned

A Literatur

B Glossar